

Ciudad de México, a 2 de octubre de 2026

Asunto: Carta de terminación de servicio social

Ma. del Carmen Silva Espinosa
Jefa de la Sección de Servicio Social
Unidad Cuajimalpa

Por este conducto hago de su conocimiento que el alumno **Oscar David González Pintor**, con número de matrícula **2173071702**, inscrito en la licenciatura de **Ingeniería en Computación**, ha concluido su servicio social en el programa denominado: *"Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo"*.

Realizando las siguientes actividades:

- Migración del entorno de simulación de ROS 2 Iron a ROS 2 Jazzy Jalisco con Gazebo Harmonic, incluyendo la reconfiguración del bridge de comunicación y controladores de articulaciones.

- Desarrollo e implementación del nodo de control autónomo en C++ que representa la Corteza Premotora de la arquitectura cognitiva del robot Xolobot.
- Configuración y calibración de sensores de contacto en el simulador, correspondientes a las Cortezas Somatosensorial y Parietal del modelo cognitivo.
- Diseño e implementación del algoritmo de trayectorias cinemáticas para el agarre reactivo de objetos.
- Pruebas de integración del sistema completo, verificando la comunicación entre el simulador, el controlador de trayectorias y los sensores de contacto.
- Generación de documentación técnica del proyecto para su continuidad y traspaso a futuros desarrolladores.

Cabe mencionar que la acreditación del mismo requirió un mínimo de 480 horas dentro de un plazo no menor a seis meses, cubriendo un horario de cuatro horas de lunes a viernes, con fecha de inicio del **1.º de abril de 2026 al 2 de octubre de 2026.**

Sin otro particular, quedo de usted.

Atentamente
Casa abierta al tiempo

Dra. Alicia Montserrat Alvarado González
Responsable del programa o proyecto
de servicio social

Oscar David González Pintor
Prestador de servicio social
Matrícula: 2173071702

Primer Reporte Trimestral — Meses 1, 2 y 3

Trimestre 26-P — Reporte de Servicio Social

Programa: Apoyo para implementar la Arquitectura Cognitiva
Inspirada en Neuronas Espejo

Alumno: Oscar David González Pintor · Matrícula: 2173071702

Licenciatura: Ingeniería en Computación — UAM Cuajimalpa

Responsable del proyecto: Dra. Alicia Montserrat Alvarado
González (Núm. económico: 41051)

Periodo reportado: 1 de abril de 2026 — 30 de junio de 2026 (Meses
1, 2 y 3)

Introducción

El presente reporte trimestral documenta las actividades realizadas durante los primeros tres meses del servicio social, inscrito bajo el programa **Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo**. Este proyecto busca dotar a un brazo robótico simulado de una arquitectura de cómputo inspirada en los principios neurocognitivos de las neuronas espejo: regiones corticales especializadas que se activan tanto al ejecutar una acción motora como al observar a otro agente ejecutarla, estableciendo la base neurológica de la imitación, el aprendizaje y la predicción motora.

El sustrato físico de la arquitectura es el brazo robótico antropomórfico **Xolobot**, un manipulador de 21 grados de libertad con una mano de cinco dedos. El trabajo de este trimestre se centró en

migrar y modernizar el entorno de simulación del Xolobot — necesario para que las cortezas cognitivas pudieran ejecutarse de forma reproducible — y en implementar los módulos computacionales que corresponden a la **Corteza Premotora**, la **Corteza Motora Primaria**, las **Cortezas Somatosensorial y Parietal**, y los primeros pasos hacia la integración del brazo y el robot simulados en un mismo espacio computacional.

Antecedentes

El proyecto heredó una versión funcional del brazo Xolobot bajo ROS 2 Iron Irwini con Gazebo Clásico en Ubuntu 22.04. Si bien el robot podía levantarse y ejecutar trayectorias básicas, el entorno de simulación no era reproducible: dependía de una configuración manual del sistema operativo y no contaba con ningún mecanismo de containerización. Esto impedía distribuir el entorno entre los integrantes del laboratorio y hacía inviable la ejecución de los módulos cognitivos en máquinas con distintas configuraciones de hardware.

Adicionalmente, existía un referente de la arquitectura cognitiva en el proyecto **Robotic-Swarms**, que implementaba el módulo de **Corteza Premotora** sobre ROS 2 Foxy y servía como caso de estudio para comprender la interfaz de los nodos cognitivos con Gazebo.

Objetivo

Apoyar la implementación de la Arquitectura Cognitiva Inspirada en Neuronas Espejo para el robot Xolobot, mediante:

1. La migración del entorno de simulación de ROS 2 Iron con Gazebo Clásico a **ROS 2 Jazzy Jalisco con Gazebo Harmonic**, garantizando su reproducibilidad mediante una imagen Docker pre-compilada de cinco capas.
2. El desarrollo del **nodo de Corteza Premotora** (SimulationController) en C++, capaz de generar trayectorias articulares reactivas y enviar su resultado a la memoria procedural del robot simulado.
3. La adaptación de las **Cortezas Somatosensorial y Parietal** al robot simulado, materializada en la configuración de siete sensores de contacto (bumpers) en Gazebo y su integración a través del bridge de comunicación.

4. La configuración de la **Corteza Motora Primaria** mediante el JointTrajectoryController de ros2_control, que traduce las órdenes del nodo cognitivo en movimiento articular ejecutado por el simulador.

5. La integración del brazo y el robot simulados en el **mismo espacio computacional**, conectando los contenedores sim (Gazebo) y brain (nodo cognitivo) a través de una red compartida y memoria IPC común.

Planeación

Calendario Oficial del Proyecto

La siguiente tabla reproduce las actividades oficiales del programa de servicio social tal como figuran en la carta de aceptación emitida el 31 de marzo de 2026. El reporte actual cubre los **Meses 1, 2 y 3**.

Actividad	Mes 1	Mes 2	Mes 3	Mes 4	Mes 5	Mes 6
Apoyo en la implementación del nodo Corteza Premotora y enviar a la memoria procedural: robot simulado	♦	♦				
Apoyo para levantar el brazo en ROS1						
Apoyo para migrar el robot de ROS1 a ROS2	♦	♦	♦	♦	♦	♦
Apoyo para adaptar lo que ya se tiene de las cortezas Parietal, Somatosensorial, motora primaria al robot simulado		♦				
Apoyo para poner en el mismo espacio el brazo y el robot simulados			♦	♦		
Apoyo en la documentación del proyecto	♦	♦	♦	♦	♦	♦
Informe trimestral			♦			♦
Informe final						♦

Correspondencia semana–mes para este trimestre:

Mes	Semana s	Actividades ejecutadas
1 (Abril)	1–4	Entorno Docker, xhost, estudio de Robotic-Swarms (referencia de Corteza Premotora)
2 (Mayo)	5–8	Arquitectura Xolobot, Corteza Motora Primaria (JTC), Cortezas Somatosensorial y Parietal (sensores)

3		Imagen Docker 5 capas, Corteza Premotora
(Junio	9-12	(SimulationController), integración
)		brain+sim, VM

Justificación de la Estrategia de Containerización con Docker

Antes de iniciar el desarrollo técnico, se evaluaron las opciones de entorno disponibles. La conclusión fue que Docker era la única solución viable para cumplir los requisitos de reproducibilidad que la arquitectura cognitiva exige: los módulos deben poder ejecutarse en cualquier equipo del laboratorio sin intervención manual.

Aislamiento

Los módulos cognitivos se desarrollan como nodos ROS 2 independientes. Docker permite ejecutar diferentes versiones de ROS 2 en contenedores distintos sin interferencias entre sus dependencias, lo que facilita el desarrollo paralelo de módulos.

Limpieza

El historial de instalaciones del laboratorio incluía residuos de paquetes incompatibles que contaminaban el entorno de ROS 2. Con Docker, cualquier estado corrupto se resuelve eliminando el contenedor y recreándolo desde la imagen, sin afectar el sistema operativo del host.

Compatibilidad

ROS 2 Jazzy Jalisco requiere Ubuntu 24.04 como base. Docker provee el sistema de archivos de Ubuntu 24.04 dentro del contenedor con independencia del sistema operativo del host.

Portabilidad para la Arquitectura Cognitiva

La imagen Docker pre-compilada garantiza que cualquier nodo cognitivo futuro (Corteza Prefrontal, Hipocampo, etc.) pueda incorporarse al sistema ejecutando un único comando de construcción.

Inicio

Al comenzar el trimestre se verificó que el entorno preexistente de ROS 2 Iron era funcional en el equipo original pero no portable. Se estableció como primer hito la obtención de un entorno Docker completamente operativo antes de proceder con la implementación de los módulos cognitivos.

Semana 1 — Preparación del entorno gráfico y diagnóstico del sistema

La arquitectura cognitiva requiere un simulador 3D para ejecutar el robot en su entorno. Gazebo Harmonic utiliza el protocolo X11 para renderizar su interfaz gráfica. El primer obstáculo técnico fue configurar el forwarding de esta interfaz hacia el contenedor Docker.

Paso A: Habilitar el acceso gráfico (crítico para Gazebo)

Sin esta configuración, Gazebo termina con el siguiente error al intentar inicializar su renderer:

```
qt.qpa.xcb: could not connect to display :0
```

```
qt.qpa.plugin: Could not load the Qt platform plugin  
"xcb"
```

```
[ERROR] [gz-5]: process has died [pid 8423, exit code  
1]
```

La solución es autorizar al demonio Docker para usar el socket X11 del host antes de lanzar cualquier contenedor:

```
xhost +local:docker
```

Paso B: Verificación de Docker Engine

```
docker --version
```

```
# Docker version 29.6.1, build 8900f1d
```

```
docker compose version
```

```
# Docker Compose version v5.3.0
```

Se confirmó que el usuario no pertenecía al grupo docker, lo que requería sudo en todos los comandos. Se corrigió con:

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

Semana 2 — Investigación: Estructura de las imágenes ROS 2 en Docker

Se estudió la organización de las imágenes oficiales de OSRF para comprender cómo construir el entorno cognitivo de forma estratificada. La jerarquía base es:

osrf/ros:jazzy-desktop

└─ Ubuntu 24.04 (Noble Numbat)

└─ ROS 2 Jazzy Jalisco (tools + desktop)

└─ Gazebo Harmonic + ros2-control + bridge

└─ workspace compilado (nodos
cognitivos Xolobot)

Se estableció el principio de **workspace pre-compilado**: la imagen compila todos los nodos cognitivos durante el docker build, haciendo que el arranque del sistema sea instantáneo. Esto es crítico para las sesiones de laboratorio, donde el tiempo de setup debe ser mínimo.

Semana 3 — Configuración del repositorio y control de versiones

Se organizó el repositorio Git con una estrategia de ramas por área funcional:

Rama	Módulo cognitivo correspondiente
main	Código estable de producción
fix/cinematica-estable	Correcciones de Corteza Motora Primaria
feat/agarre-autonomo-waypoint	Corteza Premotora — algoritmo de agarre reactivo
infra/docker-gazebo-harmonic	Infraestructura de integración (mismo espacio brain+sim)

Se implementó el script `setup_alias.sh` para automatizar la configuración del entorno en nuevas máquinas. El script es idempotente: verifica la existencia del bloque antes de escribir, evitando duplicados en `~/.bashrc`.

Semana 4 — Antecedente: Proyecto Robotic-Swarms — Referencia de la Corteza Premotora

Se estudió y ejecutó el proyecto **Robotic-Swarms**, que implementa una arquitectura cognitiva completa sobre ROS 2 para robótica de enjambre. Su paquete `cognitive_architecture` contiene nodos de planificación de acción que replican el comportamiento de la Corteza Premotora y constituyó el **referente directo** del módulo que se implementaría para el Xolobot en la Semana 10.

```
docker pull osrf/ros:foxy-desktop
```

```
docker run -it --rm --net=host \
```

```
-e DISPLAY=$DISPLAY \
```

```
-v /tmp/.X11-unix:/tmp/.X11-unix \
```

```
-v $(pwd):/root/ros2_ws/src/Robotic-Swarms-main \
```

Semana 5 — Análisis comparativo: ROS Noetic vs ROS 2 Foxy/Jazzy

Aspecto	ROS 1 Noetic	ROS 2 Foxy/Jazzy
Compilador	catkin_make	colcon build
Comunicación internodos	XMLRPC + rosmaster	DDS (Data Distribution Service)
Ciclo de vida de nodos	Ninguno	Lifecycle nodes (unconfigured → active)
Soporte a largo plazo	EOL Mayo 2025	Activo — Jazzy LTS hasta 2029
Idoneidad para arq. cognitiva	Limitada	Óptima — lifecycle permite inicialización ordenada de cortezas

Esta comparativa justificó técnicamente la decisión de desarrollar la arquitectura cognitiva del Xolobot exclusivamente sobre ROS 2 Jazzy.

Semana 6 — Estudio del proyecto base Xolobot en ROS 2 Iron

Se procedió a estudiar la versión original del Xolobot bajo ROS 2 Iron y Gazebo Clásico. Errores catalogados durante la configuración inicial:

Error	Causa	Solución
Cannot locate rosdep definition	Terminal sin source de ROS 2	Activar entorno antes de rosdep
Duplicate package names	Paquetes en src/ y src_lata/	touch src_lata/COLCON_IGNORE
Package 'xolobot_arm' not found	install/setup.bash no activado	source install/setup.bash post-build
robot_description MUST not be empty	Launch file incorrecto	Usar xolobot_arm_control.launch.py

Semana 7 — Corteza Motora Primaria — Configuración del JointTrajectoryController

En la arquitectura cognitiva, la **Corteza Motora Primaria** traduce los

planes de acción generados por la Corteza Premotora en señales motoras dirigidas a los efectores. En el sistema Xolobot, este rol lo cumple el **JointTrajectoryController (JTC)** de `ros2_control`.

Estructura articular del Xolobot (21 DOF):

Índice	Joint	Segmento
0	jnt_pecho_hombro	Rotación de base
1	jnt_hombro_hombro	Abducción lateral
2	jnt_hombro_biceps	Flexión del hombro
3	jnt_biceps_codo	Flexión del codo
4	jnt_codo_antebrazo	Pronación/ supinación
5	jnt_antebrazo_pulgar	Flexión de muñeca
6–8	Pulgar (3 falanges)	Mano
9–11	Índice (3 falanges)	Mano
12–14	Medio (3 falanges)	Mano
15–17	Anular (3 falanges)	Mano
18–20	Meñique (3 falanges)	Mano

Problema crítico resuelto — paquetes de `ros2_control` faltantes:

```
sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers
```

```
ros2 daemon stop && ros2 daemon start
```

```
ros2 control list_controllers
```

```
# joint_trajectory_controller  active
```

Semana 8 — Cortezas Somatosensorial y Parietal — Sensores de contacto y bridge

En la arquitectura cognitiva, la **Corteza Somatosensorial** procesa la información táctil del cuerpo, mientras que la **Corteza Parietal** integra esa información para construir un modelo espacial de la interacción con el entorno. Para el Xolobot simulado, estos módulos se implementan como **siete sensores de contacto (bumpers)**.

Migración de plugins SDF — diferencia crítica entre versiones de Gazebo:

```
<!-- Gazebo Clásico (Iron) -->

<plugin name="gazebo_ros_control"
filename="libgazebo_ros_control.so"/>
```

```
<!-- Gazebo Harmonic (Jazzy) – plugins nativos gz-sim  
-->
```

```
<plugin filename="gz-sim-physics-system"
```

```
name="gz::sim::systems::Physics"/>
```

```
<plugin filename="gz-sim-sensors-system"
```

```
name="gz::sim::systems::Sensors">

<render_engine>ogre2</render_engine>

</plugin>
```

Semana 9 — Integración en el mismo espacio — Imagen Docker de 5 capas

Poner el brazo y el robot simulados "en el mismo espacio" implica que los nodos cognitivos (brain) y el simulador físico (sim) puedan comunicarse como si estuvieran en la misma máquina, con latencias mínimas y descubrimiento automático de tópicos.

services:

sim: # El cuerpo: Gazebo + JTC (Corteza Motora
Primaria)

image: xolobot:jazzy

network_mode: host

command: ros2 launch xolobot_arm
xolobot_arm_control.launch.py

```
brain: # La mente: SimulationController (Corteza  
Premotora)
```

```
image: xolobot:jazzy
```

```
network_mode: host
```

```
depends_on:
```

- sim

command: >

ros2 run xolobot_arm_server xolobot_arm_server

--ros-args -p use_sim_time:=true

Semana 10 — Corteza Premotora — Implementación del nodo `SimulationController`

La **Corteza Premotora** es la región cortical responsable de la planificación y selección de acciones motoras complejas. Para el Xolobot, se implementa como el nodo C++ `SimulationController`: recibe señales de la Corteza Somatosensorial (tópicos de bumpers), planifica una secuencia de acciones (aproximación, agarre, elevación) y envía el plan resultante a la Corteza Motora Primaria (JTC).

Arquitectura de la máquina de estados reactiva:

```
[INICIO] → warm-up (4 ticks × 2.5s = 10s para  
estabilizar /clock)
```

↓

[PLANIFICACIÓN] → Corteza Premotora genera
trayectoria de 2 puntos



[EJECUCIÓN] → Corteza Motora Primaria interpola hacia
la lata



[CONTACTO] → Corteza Somatosensorial detecta colisión
palma/dedo

↓

[AGARRE] → Cierre de dedos + activación de articulación magnética

↓

[MEMORIA PROCEDURAL] → Corteza Premotora envía trayectoria de elevación

Valores de la pose de agarre (calibrados con Teach Pendant en RQT):

Joint	Valor (rad)	Descripción
jnt_pecho_hombro	0.159	Rotación del hombro hacia el objeto
jnt_hombro_hombro	0.0	Alineación lateral
jnt_hombro_biceps	-1.2874	Flexión del hombro (valor calibrado)
jnt_biceps_codo	1.5010	Extensión del codo
jnt_codo_antebrazo	-1.068	Pronación del antebrazo
jnt_antebrazo_palma	-1.1309	Flexión de muñeca (-64.8°, perpendicular al objeto)

Semana 11 — Pruebas de integración en Máquina Virtual

Al replicar el entorno en una máquina virtual para validar la portabilidad, se presentaron cuatro problemas de infraestructura que afectan directamente la capacidad de los contenedores brain y sim de cohabitar en el mismo espacio computacional.

Problema	Síntoma	Solución
Colapso gráfico de Gazebo	could not connect to display :0	~/Xauthority portable en compose
Bloqueo DDS en red de VM	/clock no visible desde brain	ROS_LOCALHOST_ONLY=1

Aislamiento entre contenedores	IPC	JTC trayectorias silenciosamente	ignoraba	ipc: host en ambos servicios
Colisión cinemática aproximación	en	Antebrazo con pedestal	chocaba	Waypoint intermedio alto

Semana 12 — Memoria Procedural — Adaptación del plan motor por retroalimentación

La solución al problema de colisión cinemática refleja el principio de **aprendizaje por retroalimentación** que define a la memoria procedural: el plan motor inicial produjo un fallo (colisión), y la secuencia fue modificada para evitarlo en ejecuciones futuras mediante un waypoint intermedio.

```
[inicio] — [waypoint_alto] — [pose_final]
```

```
vertical          (sobre pedestal)      ↓ descenso
```

Observaciones y Conclusiones

Al término del Trimestre 26-P se cumplieron los objetivos establecidos para los **Meses 1, 2 y 3**. Los módulos computacionales de la Arquitectura Cognitiva implementados en este trimestre son:

Módulo cognitivo	Implementación técnica	Semana
Corteza Premotora	SimulationController C++ — planificación y trayectorias reactivas	4 (ref.), 10
Corteza Motora Primaria	JointTrajectoryController — ejecución articular en Gazebo	7
Corteza Somatosensorial y Parietal	7 bumpers + bridge YAML — retroalimentación táctil	8
Mismo espacio brain+sim	network_mode: host + ipc: host — ubicación computacional	9, 11, 12
Memoria Procedural	Ajuste del plan motor por retroalimentación — waypoint cinemático	12

El sistema es capaz de ejecutar la secuencia completa de agarre autónomo — aproximación, contacto, cierre de dedos, elevación — en cualquier equipo del laboratorio, cumpliendo el objetivo de reproducibilidad y portabilidad que la arquitectura cognitiva exige.

Segundo Reporte Trimestral — Meses 4, 5 y 6

Trimestre 26-P — Segundo Reporte de Servicio Social

Programa: Apoyo para implementar la Arquitectura Cognitiva
Inspirada en Neuronas Espejo

Alumno: Oscar David González Pintor · Matrícula: 2173071702

Licenciatura: Ingeniería en Computación — UAM Cuajimalpa

Responsable del proyecto: Dra. Alicia Montserrat Alvarado
González (Núm. económico: 41051)

Periodo reportado: 1 de julio de 2026 — 30 de septiembre de 2026
(Meses 4, 5 y 6)

Introducción

El presente reporte trimestral documenta las actividades realizadas durante los meses cuatro, cinco y seis del servicio social, inscrito bajo el programa **Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo**. Durante este periodo, el trabajo se desplazó de la implementación inicial de los módulos cognitivos — completada en el trimestre anterior— hacia su **validación, consolidación y transferencia**.

El sustrato técnico sobre el que opera la arquitectura fue sometido a pruebas de reproducibilidad en hardware independiente, se reorganizó el repositorio oficial del proyecto para facilitar su continuidad por nuevos integrantes del laboratorio, y se ejecutó el traspaso formal del entorno a un alumno de incorporación reciente.

Antecedentes

El primer reporte trimestral (Meses 1–3, abril–junio 2026) estableció

la infraestructura computacional sobre la que opera la arquitectura cognitiva. Los logros de ese periodo fueron:

- Migración del entorno de simulación de ROS 2 Iron/Gazebo Clásico a **ROS 2 Jazzy Jalisco con Gazebo Harmonic**, encapsulado en una imagen Docker de 5 capas reproducible.
- Implementación de la **Corteza Premotora** (`SimulationController.cpp`) como nodo C++ capaz de planificar trayectorias reactivas y detectar contacto a través de 7 bumpers.
- Configuración de la **Corteza Motora Primaria** (JTC de `ros2_control`) y de las **Cortezas Somatosensorial y Parietal** (bridge YAML con Gazebo Harmonic).
- Resolución de los cuatro problemas de infraestructura en VM: forwarding gráfico X11, sincronización DDS, aislamiento IPC y colisión cinemática con waypoint.

- Validación de la secuencia completa de agarre autónomo: aproximación → contacto → cierre de dedos → elevación.

Objetivo

Consolidar y transferir la Arquitectura Cognitiva Inspirada en Neuronas Espejo para el robot Xolobot, mediante:

1. La validación de la reproducibilidad del entorno en hardware independiente, mediante pruebas de estrés en una instalación limpia de Ubuntu 24.04.
2. La reorganización del repositorio oficial del proyecto en un repositorio dedicado (xolobot-cognitive-core), con documentación maestra que garantice la incorporación autónoma de nuevos integrantes.

3. La gestión de la infraestructura de desarrollo: resolución de conflictos de dependencias, optimización del almacenamiento en disco y expansión de la capacidad de almacenamiento disponible.

4. El traspaso formal del entorno al alumno Diego Vázquez, en coordinación con la Dra. Montserrat.

Planeación

Calendario Oficial del Proyecto

La siguiente tabla reproduce las actividades oficiales del programa de servicio social. El reporte actual cubre los **Meses 4, 5 y 6**.

Actividad	Mes 1	Mes 2	Mes 3	Mes 4	Mes 5	Mes 6
Apoyo en la implementación del nodo Corteza Premotora	♦	♦				
Apoyo para migrar el robot de ROS1 a ROS2	♦	♦	♦	♦	♦	♦
Apoyo para adaptar cortezas Parietal, Somatosensorial, motora primaria		♦				
Apoyo para poner en el mismo espacio el brazo y el robot simulados			♦	♦		
Apoyo en la documentación del proyecto	♦	♦	♦	♦	♦	♦
Informe trimestral			♦			♦
Informe final						♦

Correspondencia semana–mes para este trimestre:

Mes	Semanas	Actividades ejecutadas
4 (Julio)	13–16	Validación Docker en VM, reorganización del repositorio oficial, documentación maestra
5 (Agosto)	17–20	Prueba Acid Test en ss-neuro, resolución de conflictos de dependencias, gestión de almacenamiento
6 (Septiembre)	21–24	Expansión de almacenamiento NTFS, documentación de handover, onboarding de Diego Vázquez, informe final

Inicio

El trimestre comenzó con el entorno cognitivo funcionando sobre el equipo principal de desarrollo. El reto de esta fase no era ya implementar, sino **garantizar que lo implementado pudiera sobrevivir fuera de ese equipo**: en una máquina diferente, con un usuario nuevo, sin la memoria contextual del desarrollador original.

Desarrollo

Semanas 13–15 — Consolidación de la integración brain+sim en Docker

Los cuatro problemas resueltos en el trimestre anterior fueron verificados y confirmados como estables:

Problema	Síntoma original	Solución aplicada	Estado
Colapso gráfico de Gazebo	could connect to display :0	~/Xauthority portable en compose	✓ Estable
Bloqueo DDS en red de VM	/clock no visible desde brain	ROS_LOCALHOST_ONLY=1	✓ Estable
Aislamiento IPC entre contenedores	JTC ignoraba trayectorias silenciosamente	ipc: host en ambos servicios	✓ Estable
Colisión cinemática en aproximación	Antebrazo chocaba con pedestal	Waypoint alto intermedio	✓ Estable

Se ejecutó el flujo completo desde imagen limpia para confirmar que no existían dependencias ocultas del entorno del desarrollador. La secuencia completa de agarre autónomo se ejecutó correctamente desde la imagen recién construida.

Semanas 16–18 — Repositorio oficial: `xolobot-cognitive-core`

Se creó el repositorio `xolobot-cognitive-core` bajo la organización del alumno en GitHub y se estableció como repositorio principal del programa. El repositorio anterior fue marcado con `COLCON_IGNORE` y se eliminó su remote, preservándolo solo como referencia histórica local.

Resolución del conflicto de versiones de libfastcdr

apt upgrade actualizó libfastcdr de la versión 2.2.7 a 2.2.8. El directorio build/ contenía entradas de CMakeCache.txt con la ruta codificada a la versión anterior, que ya no existía. Solución: reconstrucción limpia eliminando los artefactos de build previos.

```
cd ~/migration_ws
```

```
rm -rf build/ install/ log/
```

```
colcon build
```

```
source install/setup.bash
```

Aprendizaje documentado: Después de ejecutar `apt upgrade`, es necesario realizar un rebuild limpio. Esto aplica también a entornos del laboratorio que reciban actualizaciones automáticas.

README maestro y documentación de arquitectura

Se redactó el `README.md` maestro del repositorio con la estructura definitiva para nuevos integrantes, cubriendo: prerequisites del sistema, instalación paso a paso, panel de alias de control, opción nativa y Docker. Como complemento, se generó un documento técnico de apoyo para el traspaso del proyecto (`DEV_CONTEXT.md`), que consolida la arquitectura del sistema, el estado actual del código y las decisiones de diseño relevantes para quien continúe el desarrollo.

La pregunta que define este bloque es directa: **¿puede cualquier integrante del laboratorio, partiendo de una máquina limpia, llegar a tener el sistema corriendo siguiendo solo el README?**

Configuración del equipo de prueba

Se habilitó una laptop denominada ss-neuro como equipo de prueba independiente, con instalación limpia de Ubuntu 24.04 LTS, hardware con recursos moderados y sin historial de ROS 2 en el sistema.

[Insertar imagen: Laptop ss-neuro durante la instalación de Ubuntu 24.04 — equipo de prueba sin

configuración previa]

Prueba de estrés — colcon build en hardware limitado

La compilación tomó entre **2 y 3 horas**, manteniendo todos los núcleos de la CPU a su máxima capacidad de forma sostenida. Este comportamiento es esperado y no indica un error: el ecosistema ROS 2 Jazzy con Gazebo Harmonic requiere compilar cientos de unidades de traducción con dependencias encadenadas.

[Insertar imagen: Monitor del sistema mostrando CPU al 100% durante colcon build en ss-neuro — prueba

de estrés de 2-3 horas]

Validación final — Gazebo Harmonic en ejecución

Gazebo Harmonic renderizó el brazo robótico Xolobot, el pedestal y la lata sin errores. El nodo `SimulationController` se sincronizó con el tiempo de simulación y completó la secuencia de agarre autónomo.

[Insertar imagen: Clonación del repositorio xolobot-cognitive-core en ss-neuro — terminal mostrando git

clone exitoso]

[Insertar imagen: Gazebo Harmonic ejecutándose en ss-neuro — brazo Xolobot y lata renderizados

correctamente]

Conclusión del Acid Test: El entorno es **reproducible**. Un usuario nuevo, partiendo de Ubuntu 24.04 limpio y siguiendo la documentación del repositorio, llega a tener el sistema corriendo sin necesidad de asistencia del desarrollador original.

Semanas 22–24 — Gestión de infraestructura y traspaso formal

Por qué el proyecto demanda tanto almacenamiento

La partición principal de Ubuntu llegó al **97% de ocupación**, con solo 5 GB libres de los 198 GB asignados. Las cinco capas que componen el ecosistema de dependencias explican este consumo:

Capa	Espacio aproximado	Descripción
ROS 2 Jazzy Desktop (<code>/opt/ros/jazzy/</code>)	3–4 GB	Framework DDS, <code>rviz2</code> , <code>rqt</code> , herramientas CLI y dependencias
Gazebo Harmonic (<code>gz-harmonic</code>)	4–6 GB	Motor de física (Bullet/DART), renderizado Ogre2, modelos 3D base
Caché de compilación (<code>build/</code>)	1–2 GB	Archivos objeto, <code>CMakeCache.txt</code> , artefactos de Ninja — regenerables
Imágenes Docker (<code>/var/lib/docker/</code>)	8–12 GB por imagen	Contiene todo lo anterior pre-compilado; varias versiones acumulan más
Logs de ejecución (<code>~/ros/log/</code>)	0.5–2 GB	Crece con cada sesión de trabajo; se puede limpiar con <code>rm -rf ~/ros/log/</code>

Gestión del espacio en disco — recuperación de 13 GB

Se liberaron aproximadamente 13 GB, llevando la partición del 97% al 81% de ocupación con 37 GB libres. Los principales orígenes liberados fueron: workspace ROS 2 compilado desde fuente (11 GB, ya no necesario), logs de ejecuciones pasadas (1.2 GB) y caché de apt (1 GB).

Expansión de almacenamiento — Partición NTFS como volumen auxiliar

Se eligió montar permanentemente la partición NTFS con `ntfs-3g`, haciendo disponibles 529 GB como volumen auxiliar en `/mnt/datos`, sin modificar la tabla de particiones ni arriesgar la integridad del sistema Linux.

```
UUID=124A39244A39064F /mnt/datos ntfs-3g \
defaults,uid=1000,gid=1000,umask=022,nofail 0 0
```

Traspaso formal — Onboarding de Diego Vázquez

Fecha de inicio: 28 de septiembre de 2026

En coordinación con la Dra. Montserrat, se inició el traspaso del entorno actualizado al alumno Diego Vázquez. Se detectó que Diego contaba con Ubuntu 22.04, versión incompatible con ROS 2 Jazzy Jalisco. Se acordó la creación de una partición dedicada para instalar Ubuntu 24.04, garantizando compatibilidad nativa con el stack cognitivo del proyecto.

Materiales entregados: URL del repositorio oficial, enlace a la documentación oficial de instalación de ROS 2 Jazzy, e instrucción de lectura previa del README.md maestro antes de ejecutar cualquier comando.

Observaciones y Conclusiones

Al término de los Meses 4, 5 y 6 del calendario oficial, el proyecto completó su transición de la fase de implementación a la fase de **validación y transferencia**.

Actividad	Resultado	Semanas
Validación Docker en VM	Todos los problemas confirmados como estables; ciclo completo de arranque verificado	13–15
Reorganización del repositorio	xolobot-cognitive-core en GitHub, documentación maestra, onboarding autónomo posible	16–18
Resolución de conflicto libfastcdr	Rebuild limpio documentado como procedimiento estándar post-apt upgrade	16–18
Acid Test en ss-neuro	Gazebo Harmonic ejecutándose en instalación limpia Ubuntu 24.04; secuencia de agarre completada	19–21
Gestión de almacenamiento	Liberación de 13 GB (97% → 81%); 529 GB auxiliares disponibles en /mnt/datos	22–24
Traspaso a Diego Vázquez	Prerrequisitos diagnosticados, materiales entregados, onboarding iniciado	22–24

El estado del proyecto al cierre del servicio social es el siguiente: la arquitectura cognitiva del Xolobot opera en ROS 2 Jazzy Jalisco con Gazebo Harmonic, su documentación permite la incorporación

autónoma de nuevos integrantes, y el proceso de traspaso a la siguiente generación de alumnos ha sido formalmente iniciado con la orientación de la responsable del proyecto.